

ระบบตรวจสอบเอกสาร PDF ด้วย AI

File Checker

การไฟฟ้าฝ่ายผลิตแห่งประเทศไทย

53 หมู่ 2 ถนนเจริญสุขนิทวงศ์ ตำบลบางกรวย อำเภอบางกรวย จังหวัดนนทบุรี 11130

จัดทำโดย

นายศิลาชัย เกษตรกุลชีพ

นักศึกษาฝึกงาน

แผนก หวข-ฟ กอง กวป-ฟ ฝ่ายบริหารจัดการสินทรัพย์ผลิตไฟฟ้า

1. ภาพรวมระบบ

ระบบ File Checker เป็นเว็บแอปพลิเคชันที่ช่วยให้ผู้ใช้สามารถตรวจสอบเอกสาร PDF มีเนื้อหาครบถ้วนตามรายการตรวจสอบ (Checklist) ที่กำหนดเองหรือไม่ โดยใช้ปัญญาประดิษฐ์ (AI) ในการวิเคราะห์

ผลลัพธ์ที่ได้คือรายงานแสดงผลในรูปแบบของไฟล์ Excel พร้อมหลักฐานอ้างอิงจากเอกสาร โดยระบบรองรับการเลือกใช้ AI ทั้งแบบประมวลผลภายในเครื่อง (Local) และแบบ Cloud

ระบบรองรับ AI สามรูปแบบ

- Local AI: Gemma 4 ผ่าน Ollama ทำงานบนเครื่องทั้งหมด ข้อมูลไม่ออกนอกองค์กร เหมาะกับเอกสารที่มีความลับ
- EGAT AI Gateway: API จากฝ่ายกลยุทธ์เทคโนโลยีดิจิทัล
- Cloud AI: Google Gemini API และ OpenAI API

ผลลัพธ์ที่ได้คือรายงาน ผ่าน/ไม่ผ่าน รายชื่อ ในรูปแบบของไฟล์ Excel พร้อมหลักฐานอ้างอิงจากเอกสาร

2. ที่มาและความสำคัญ

2.1 ปัญหาเดิม

- ใช้เวลานาน การตรวจสอบเอกสารด้วยบุคลากรแต่ละฉบับต้องใช้เวลาอ่านและเปรียบเทียบเป็นเวลานาน
- โอกาสผิดพลาดสูง การตรวจสอบด้วยสายตาอาจพลาดรายละเอียดสำคัญ โดยเฉพาะเอกสารที่มีจำนวนหน้ามาก
- ไม่มีมาตรฐานกลาง เกณฑ์การตรวจสอบอาจแตกต่างกันไปตามผู้ตรวจแต่ละคน ขาดความสม่ำเสมอ

2.2 แนวทางแก้ไข

นำ AI มาช่วยตรวจสอบอัตโนมัติ ระบบรับไฟล์ PDF และรายการตรวจสอบ (Checklist) จากผู้ใช้

จากนั้นใช้ปัญญาประดิษฐ์วิเคราะห์ว่าเอกสารมีเนื้อหาครบตามข้อกำหนดหรือไม่ พร้อมให้คะแนนและหลักฐานอ้างอิง

3. ฟังก์ชันหลัก

ฟังก์ชันที่ 1 การอ่านข้อความจากเอกสาร PDF

ระบบรองรับ PDF 2 ประเภท

- PDF ดิจิทัล เอกสารที่สร้างจากคอมพิวเตอร์ เช่น ส่งออกจาก Word หรือ Excel มีข้อความฝังอยู่ในไฟล์ ระบบใช้ PyMuPDF ดึงข้อความโดยตรง เปรียบเสมือนการคัดลอกข้อความจากหน้าจอ
- PDF จากการสแกน เอกสารที่ได้จากเครื่องสแกนหรือภาพถ่าย ไม่มีข้อความฝังอยู่ มีเพียงรูปภาพของตัวหนังสือ ระบบแปลงหน้าเป็นภาพแล้วใช้ Tesseract OCR อ่านข้อความภาษาไทยและอังกฤษ

ระบบมีเกณฑ์การจำแนกอัตโนมัติ หากหน้าใดมีข้อความน้อยกว่า 100 ตัวอักษร ระบบจะถือว่าเป็นภาพสแกนและส่งให้ OCR ประมวลผลโดยอัตโนมัติ

พีเจอร์ที่ 2 การแบ่งข้อความ (Chunking)

AI แต่ละโมเดลมีขีดจำกัดปริมาณข้อความที่รับได้ต่อครั้ง (Context Window) หากเอกสารมีขนาดใหญ่เกินไป ระบบจะแบ่งเป็นส่วนย่อยก่อน

- Chunk Size 24,000 ตัวอักษร (ประมาณ 6,000–8,000 คำ)
- Chunk Overlap 2,000 ตัวอักษร เพื่อป้องกันเนื้อหาที่อยู่ตรงรอยต่อสูญหาย

ระบบจะส่งข้อความทีละ Chunk ให้ AI วิเคราะห์ และหากพบว่ารายการนั้น "ผ่าน" ใน Chunk ใดแล้ว จะหยุดทันทีโดยไม่ตรวจสอบ Chunk ที่เหลือ

พีเจอร์ที่ 3 การวิเคราะห์ด้วย AI

ระบบสร้างคำสั่ง (Prompt) ที่มีโครงสร้างชัดเจน ส่งให้ AI ตัดสินทีละรายการ โดยกำหนดเกณฑ์อย่างเข้มงวด สิ่งที่ส่งให้ AI ทุกครั้ง ได้แก่

- บทบาท "ผู้ตรวจสอบเอกสารที่เข้มงวด"
- ข้อกำหนดที่ต้องตรวจ (จาก Checklist)
- ข้อความจากเอกสาร (แต่ละ Chunk)
- เกณฑ์ตัดสิน 9 ข้อที่ต้องปฏิบัติตามอย่างเคร่งครัด
- รูปแบบคำตอบ JSON ที่กำหนดไว้ชัดเจน ได้แก่ สถานะ (ผ่าน/ไม่ผ่าน), เหตุผล, หลักฐานจากเอกสาร

พีเจอร์ที่ 4 รองรับ AI หลายผู้ให้บริการ

รองรับ 3 ช่องทาง เลือกได้ตามความเหมาะสม

- Local AI (Ollama และ Gemma 4 26B) ประมวลผลภายในเครื่อง เหมาะกับเอกสารที่มีความลับ ไม่มีค่าใช้จ่ายต่อครั้ง ใช้งานออฟไลน์ได้ แต่ต้องการคอมพิวเตอร์สเปคสูง
- EGAT AI Gateway ตอบเร็ว แม่นยำสูง ใช้กับเครื่องสเปคต่ำได้ รองรับโมเดล GPT 5 mini และ Gemini 2.5 Flash
- Cloud AI ตอบเร็ว แม่นยำสูง ใช้กับเครื่องสเปคต่ำได้ รองรับโมเดลจาก Gemini และ OpenAI เช่น Gemini 3.1 Pro, Gemini 3 Flash, GPT-4o, GPT-5 และสามารถเพิ่มโมเดลเองผ่าน UI ได้

คำแนะนำ: เอกสารลับภายในองค์กรให้ใช้ Local AI เอกสารทั่วไป ต้องการความเร็วและแม่นยำใช้ EGAT AI Gateway หรือ Cloud AI

พีเจอร์ที่ 5 กลไกรับมือข้อผิดพลาดอัตโนมัติ

- **Exponential Backoff Retry** เมื่อ Gemini หรือ OpenAI ตอบกลับด้วยรหัส 429 (เกินโควตา) หรือ 503 (เซิร์ฟเวอร์เต็ม) ระบบจะรอแล้วลองใหม่โดยเพิ่มเวลารอเป็นทวีคูณ สูงสุด 5 ครั้ง ได้แก่ 1 วินาที, 2 วินาที, 4 วินาที, 8 วินาที, 16 วินาที ตามลำดับ
- **JSON Fallback Parser** หาก AI ส่ง JSON ที่ไม่สมบูรณ์ (เช่น ถูกตัดกลางคัน) ระบบจะใช้ Regex แยกค่าแต่ละฟิลด์ ออกมาทีละตัว เพื่อให้ได้ผลลัพธ์ออกมาแม้ JSON จะผิดรูปแบบ

- **Thai Language Retry** ตรวจสอบว่าคำตอบที่ AI ตอบเป็นภาษาไทยหรือไม่ (ต้องมีตัวอักษรไทยอย่างน้อย 15%) หากเป็นภาษาอังกฤษ ระบบจะส่งคำสั่งใหม่ด้วย Prompt ที่เน้นย้ำกว่าเดิม
- **Sanitize Reasoning** บางครั้ง AI อาจสร้างข้อความซ้ำวนไปมา ระบบจะตรวจจับและตัดส่วนที่ซ้ำออกอัตโนมัติ ทั้งระดับตัวอักษรและระดับคำ
- **Pydantic Validation** ตรวจสอบโครงสร้าง Checklist ก่อนประมวลผล หากไม่ถูกต้องจะแจ้งกลับทันทีพร้อมรายละเอียดข้อผิดพลาด

ฟีเจอร์ที่ 6 การแสดงผลแบบ Real-time (SSE)

ผู้ใช้ไม่ต้องรอนกว่าการตรวจสอบทั้งหมดจะเสร็จสิ้น ผลลัพธ์แต่ละรายการจะปรากฏบนหน้าจอทันทีที่ AI วิเคราะห์เสร็จ ผ่านกลไก Server-Sent Events (SSE)

1. Backend ส่ง Event "start" แจ้งจำนวนรายการทั้งหมด
2. ส่ง Event "item" ทุกครั้งที่ AI ตรวจเสร็จ 1 รายการ
3. Frontend รับ Event แล้วแสดงผลทันที
4. ส่ง Event "done" พร้อมสรุปคะแนนเมื่อเสร็จทั้งหมด

ฟีเจอร์ที่ 7 การส่งออกผลลัพธ์เป็น Excel

สร้างไฟล์ .xlsx ด้วย openpyxl ประกอบด้วย 2 ชีต

- ชีตที่ 1 สรุปผลต่างๆ ได้แก่ ชื่อไฟล์, วันที่วิเคราะห์, จำนวนรายการ, จำนวนผ่าน/ไม่ผ่าน, คะแนนรวม
- ชีตที่ 2 รายละเอียดทุกข้อพร้อมสถานะ (FOUND / MISSING), เหตุผล และหลักฐานจากเอกสาร

ฟีเจอร์ที่ 8 ระบบจัดการ Checklist และ Preset

- เพิ่ม/ลบหัวข้อหลักได้ไม่จำกัด แต่ละหัวข้อมีรายการย่อยได้หลายข้อ ผู้ใช้กำหนดคะแนนเต็มของแต่ละรายการย่อยได้เอง
- บันทึก Checklist เป็น Preset ตั้งชื่อได้ โหลดกลับมาใช้ได้ทันที ลบ Preset ที่ไม่ใช้แล้วได้ จัดเก็บใน Local Storage ของเบราว์เซอร์

ฟีเจอร์ที่ 9 ระบบบันทึกผลลัพธ์

- บันทึกประวัติการตรวจสอบเพื่อให้ผู้ใช้สามารถดูย้อนหลังได้

4. สถาปัตยกรรมระบบ

4.1 ภาพรวม

แอปพลิเคชันใช้ FastAPI (ทำงานบน Uvicorn) เป็น Backend web framework ทำหน้าที่ให้บริการทั้ง API endpoint และ UI ส่วน UI สร้างด้วย HTML, CSS และ Vanilla JavaScript โดยใช้ Jinja2 templates

สำหรับการวิเคราะห์ด้วย AI รองรับ 3 ช่องทาง

- Ollama (Local) ที่ localhost:11434
- EGAT AI Gateway
- Cloud AI

โดยผู้ใช้เลือก Model ได้ผ่าน UI ส่วน Backend จะเรียกใช้งานผ่าน httpx ทั้งหมด โมเดล Local AI ที่ใช้คือ Gemma 4 (26B) จาก Google

การดึงข้อความจาก PDF ดำเนินการโดยใช้ PyMuPDF (Python wrapper สำหรับ MuPDF) ซึ่งอ่านแต่ละหน้าและดึงข้อความออกมาสำหรับหน้าที่เป็นภาพสแกนและไม่มีข้อความดิจิทัล แอปจะใช้ Tesseract OCR ผ่าน pytesseract และ Pillow ซึ่งรองรับทั้งภาษาไทยและภาษาอังกฤษ

4.2 โครงสร้างโค้ด

```
EGAT-File-Checker/
├── run.py                ← entry point (EXE / python run.py)
├── main.py              ← FastAPI app + API routes
├── config.py            ← all tuneable constants (URLs, timeouts, model
│                        name, chunk sizes)
├── build.bat            ← PyInstaller build script
├── requirements.txt
├──
├── checker/
│   ├── models.py        ← Pydantic request/response schemas
│   ├── pdf_extractor.py ← PDF to plain text (native + OCR fallback)
│   ├── orchestrator.py  ← LLM prompt logic, chunking, result parsing
│   ├── excel_exporter.py ← build .xlsx download response
│   └── history.py        ← SQLite-backed history persistence
├──
├── static/
│   ├── css/style.css
│   └── js/app.js         ← frontend logic (fetch/analyze, render results)
├──
├── templates/
│   ├── index.html       ← single-page UI served by Jinja2
│   └── history.html      ← history browser page
├──
└── docs/
    ├── index.html       ← project presentation slides (open in browser)
    └── deck-stage.js     ← slide deck controller
```

4.3 เทคโนโลยีที่ใช้

ส่วน	เทคโนโลยี
Backend	Python, FastAPI, Uvicorn
Frontend	HTML5, CSS3, Vanilla JavaScript
Template Engine	Jinja2
อ่าน PDF	PyMuPDF (fitz)
OCR (ภาพสแกน)	Tesseract OCR + Pillow
Local AI	Ollama + Gemma 4 (26B)
HTTP Client	httpx (async)
Data Validation	Pydantic v2
Excel Export	openpyxl
Distribution	PyInstaller

5. วิธีการทำงานของระบบ

5.1 แผนภาพแสดงการทำงานของระบบ



<https://kasets.art/Vt1jv9>

5.2 ขั้นตอนการวิเคราะห์เอกสาร

1. ผู้ใช้กรอก Checklist (หัวข้อและคะแนน) และอัปโหลด PDF
2. ระบบอ่านข้อความจาก PDF ใน Memory
 - หน้าที่มีข้อความ ≥ 100 ตัวอักษร จะดึงข้อความโดยตรง (เหมือน Copy-Paste)
 - หน้าที่มีข้อความ < 100 ตัวอักษร จะถือว่าเป็นภาพสแกน ใช้ Tesseract OCR อ่านแทน
3. ตรวจสอบ Checklist ทีละข้อ
 - แบ่งเอกสารเป็น Chunk ขนาด 24,000 ตัวอักษร (Overlap 2,000 ตัวอักษร)
 - ส่ง Chunk และข้อกำหนดให้ AI
 - ถ้าพบหลักฐาน (Pass) จะหยุดค้น Chunk ถัดไปทันที
 - ถ้าไม่พบทุก Chunk จะแสดงผล Fail
4. รวมผลทุกข้อ คำนวณคะแนน และเปอร์เซ็นต์ความครบถ้วน
5. แสดงผลในเบราว์เซอร์ และสามารถส่งออกเป็นไฟล์ Excel ได้

5.3 Prompt ที่ใช้กับ AI

AI ทุกโมเดลได้รับ Prompt ภาษาไทยที่กำหนดให้ตอบกลับเป็นรูปแบบ JSON และตอบเป็นภาษาไทยเท่านั้น โดยมีเกณฑ์การตัดสินดังนี้

- Pass เมื่อเอกสารมีเนื้อหาที่ระบุถึงข้อกำหนดอย่างชัดเจน มีหลักฐานอ้างอิงได้
- Fail เมื่อเอกสารไม่มีเนื้อหาที่กล่าวถึงข้อกำหนด หรือกล่าวแบบผ่านๆ ไม่มีสาระสำคัญ

ระบบยังตรวจสอบว่า AI ตอบกลับมาเป็นภาษาไทยหรือไม่ ถ้าไม่ใช่จะ Retry อัตโนมัติพร้อมคำสั่งที่เข้มงวดกว่าเดิม

5.4 รูปแบบผลลัพธ์

JSON (API Response):

```
{
  "summary": {
    "total": 10, "passed": 7, "failed": 3,
    "total_score": 100, "passed_score": 70
  },
  "similarity_score": 70,
  "results": [{
    "section": "บทสรุปผู้บริหาร",
    "items": [{
      "requirement": "สรุปผลการบริหารความเสี่ยง",
      "score": 10, "status": "pass",
      "reasoning": "พบเนื้อหาครอบคลุมในหน้า 2-3",
      "evidence": "...ผลการบริหารความเสี่ยงประจำปี 2569..."
    }]
  }]
}
```


รายงาน Excel (.xlsx)

- ซีตที่ 1 "สรุปผล" แสดงชื่อไฟล์, วันที่วิเคราะห์, จำนวนผ่าน/ไม่ผ่าน, คะแนนรวม, ร้อยละความครบถ้วน

	A	B
1	ชื่อไฟล์	บทสรุปแผนงาน มาตรการเพื่อการจัดการความเสี่ยง อฟม ประจำปี 2569.pdf
2	วันที่ตรวจสอบ	28/05/2026 11:34:58
3	จำนวนรายการทั้งหมด	3
4	ผ่าน (FOUND)	2
5	ไม่ผ่าน (MISSING)	1
6	คะแนนที่ได้	2 / 3
7	เปอร์เซ็นต์ความครบถ้วน	67%

- ซีตที่ 2 "ผลการตรวจสอบ" แสดงรายละเอียดแต่ละข้อ พร้อมหลักฐานและเหตุผล

	A	B	C	D	E	F	G
1	หัวข้อหลัก	รายการข้อบกพร่อง	คะแนนเต็ม	คะแนนที่ได้	ผล	เหตุผล	หลักฐาน
2	บทสรุปผู้บริหาร	สรุปผลการบริหารความเสี่ยง	1	1	FOUND	เอกสารมีเนื้อหาที่ระบุถึงสรุปผลการบริหารความเสี่ยงประจำปี 2568 และรายละเอียดของปัจจัยเสี่ยงด้านการดำเนินการและด้านการปฏิบัติตามกฎระเบียบ	สรุปผลการบริหารความเสี่ยง อฟม. ประจำปี 2568 ประกอบไปด้วยปัจจัยเสี่ยง 2 ด้าน ได้แก่ ด้านการดำเนินการ และ การปฏิบัติตามกฎระเบียบ
3	บทสรุปผู้บริหาร	อธิบายความเชื่อมโยงระหว่างปัจจัยเสี่ยงระหว่างปีปัจจุบัน (2569) กับปีก่อนหน้า (2568) โดยระบุผลการบริหารความเสี่ยงในไตรมาส 4-2568 เปรียบเทียบกับระดับ	1	0	MISSING		
4	การจัดการความเสี่ยง	ระบุแนวทางการตอบสนองต่อความเสี่ยง พร้อมแผนงาน/มาตรการควบคุมที่มีอยู่ (Existing Control) และแผนงาน/มาตรการบริหารจัดการความเสี่ยง (Mitigation Plan)	1	1	FOUND	เอกสารมีการระบุถึงแผนงาน/มาตรการควบคุมที่มีอยู่ (Existing Control) และแผนงาน/มาตรการบริหารจัดการความเสี่ยง (Mitigation Plan) ครบถ้วนตามข้อกำหนด	ตารางที่ 3 แผนงาน/มาตรการควบคุมที่มีอยู่ (Existing Control) และแผนงานมาตรการบริหารจัดการความเสี่ยง (Mitigation Plan)

6. ความต้องการของระบบ (System Requirements)

6.1 สำหรับการใช้งานผ่าน EGAT AI Gateway หรือ Cloud AI

รายการ	ข้อกำหนด
ระบบปฏิบัติการ	Windows 10 / 11
RAM	4 GB ขึ้นไป (8 GB แนะนำ)
พื้นที่ว่าง	500 MB
อินเทอร์เน็ต	ต้องการ (สำหรับ Gemini / OpenAI)

6.2 สำหรับการใช้งานแบบ Offline (Local AI)

รายการ	ขั้นต่ำ	แนะนำ
RAM	16 GB	32 GB ขึ้นไป
พื้นที่ว่าง	20 GB	30 GB ขึ้นไป
GPU VRAM	ไม่บังคับ (ใช้ CPU ได้)	8 GB ขึ้นไป

6.3 สำหรับ PDF ที่เป็นภาพสแกน (OCR)

ต้องติดตั้ง Tesseract OCR เพิ่มเติม พร้อม Language Pack ภาษาไทย หากไม่ติดตั้ง ระบบยังทำงานได้ปกติกับ PDF ที่มีข้อความดิจิทัล

7. ประเมินการปริมาณการใช้ Token ผ่าน EGAT AI Gateway

โครงสร้าง Prompt ต่อการตรวจสอบ 1 ข้อกำหนด ประกอบด้วย คำสั่งระบบและเกณฑ์การตัดสินใจ (System Instruction) หัวข้อและข้อกำหนดจาก Checklist เนื้อหาเอกสาร PDF ที่ถูกแบ่ง Chunk ตามขอบเขตหน้า (Page-level Chunking) และคำสั่งให้ตอบกลับในรูปแบบ JSON ที่ระบุสถานะ (pass/fail) เหตุผลภาษาไทย และข้อความหลักฐานจากเอกสาร

ผลการคำนวณจากข้อมูลตัวอย่าง (เอกสารแผนงานบริหารความเสี่ยง 1 ฉบับ ประมาณ 40 หน้า ตรวจสอบกับ Checklist 20 ข้อ)

รายการ	ปริมาณ
จำนวนข้อกำหนดที่ตรวจสอบ	20 ข้อ
Input Token ต่อเอกสาร 1 ฉบับ	ประมาณ 320,000 Tokens
Output Token ต่อเอกสาร 1 ฉบับ	ประมาณ 10,000 Tokens

หมายเหตุ: ระบบใช้กลไก Early-exit เมื่อพบหลักฐานในเอกสารแล้วจะหยุดประมวลผล Chunk ถัดไปทันที จำนวน Token จริงอาจต่ำกว่าค่าประมาณการนี้

การประมาณการระดับโครงการต่อปี

ดำเนินการตรวจสอบเอกสารแผนงานบริหารความเสี่ยงของโรงไฟฟ้าจำนวน 16 โรง ปีละ 1 ครั้ง และเพื่อรอบทบทวนภายหลังโรงไฟฟ้าปรับแก้เอกสารและส่งกลับมาให้ตรวจสอบใหม่อีก 1-2 ครั้ง โดยการคำนวณครั้งนี้เป็นการคำนวณการใช้งาน 3 ครั้งต่อปี

รายการ	ปริมาณ
จำนวนการตรวจสอบเอกสารรวมต่อปี	48 ฉบับ (16 × 3)
Input Token รวมต่อปี	ประมาณ 15,360,000 Tokens
Output Token รวมต่อปี	ประมาณ 480,000 Tokens

รายการ	Token	อัตรา	ค่าใช้จ่าย (USD)	ค่าใช้จ่าย (THB)
Input	15,360,000	\$0.25 / 1M	\$ 3.84	134.40 บาท
Output	480,000	\$2 / 1M	\$ 0.96	33.60 บาท
รวมทั้งสิ้นต่อปี	15,840,000		\$ 4.80	168 บาท

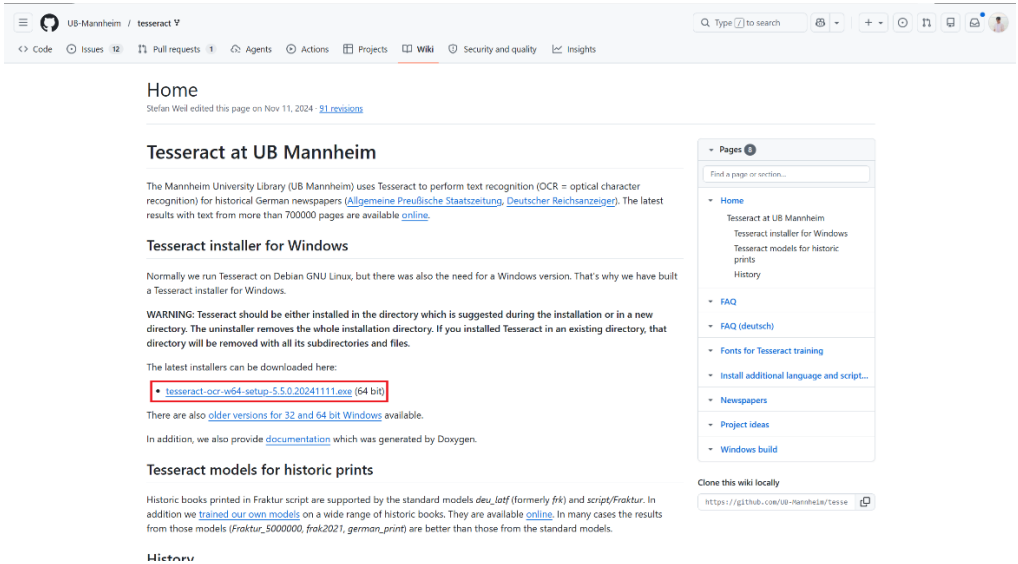
หมายเหตุ: คำนวณโดยใช้ราคาของโมเดล GPT 5 mini ณ วันที่ 22 พฤษภาคม พ.ศ. 2569 และอัตราแลกเปลี่ยนที่ 35 บาท ต่อ 1 USD

8. วิธีติดตั้งโปรแกรม

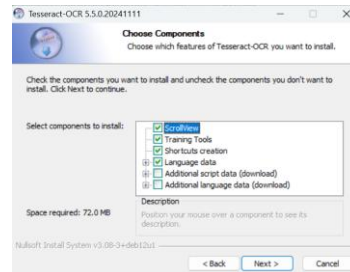
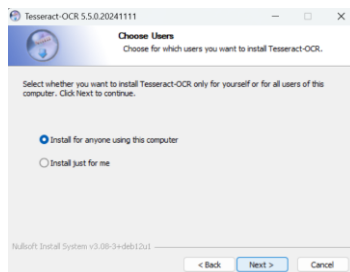
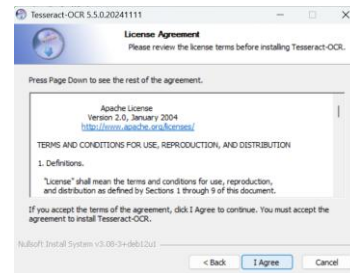
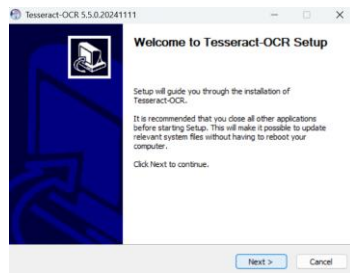
8.1 ขั้นตอนที่ 1 ติดตั้ง Tesseract OCR (สำหรับ PDF ที่เป็นรูปภาพ)

Tesseract ช่วยให้โปรแกรมอ่านข้อความจาก PDF ที่เป็นภาพสแกน

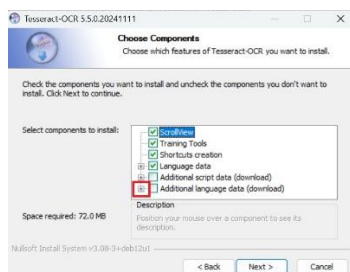
- 1. เปิดเบราว์เซอร์ไปที่ <https://github.com/UB-Mannheim/tesseract/wiki>
- 2. ดาวน์โหลดไฟล์ tesseract-ocr-w64-setup-*.exe (เวอร์ชันล่าสุด)



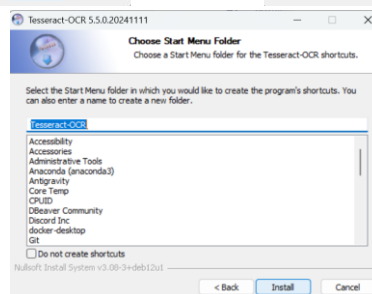
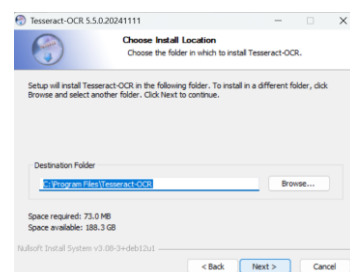
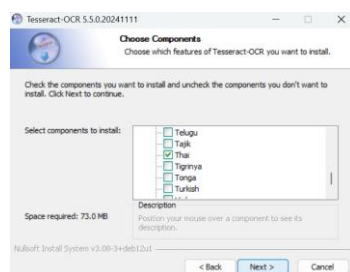
- 3. เปิดไฟล์ติดตั้ง กด Next จนถึงหน้า "Choose Components"



4. ขยาย "Additional language data" แล้วติ๊กถูกที่ "Thai"



5. กด Next ไปเรื่อยๆ แล้วกด Install



6. ตรวจสอบโดยเปิด Command Prompt แล้วพิมพ์ tesseract --version



```
C:\WINDOWS\system32>tesseract --version
tesseract v5.2.0.20220712
leptonica-1.78.0
libgif 5.1.4 : libjpeg 8d (libjpeg-turbo 1.5.3) : libpng 1.6.34 : libtiff 4.0.9 :
zlib 1.2.11 : libwebp 0.6.1 : libopenjp2 2.3.0
Found AVX2
Found AVX
Found FMA
Found SSE4.1
Found libarchive 3.5.0 zlib/1.2.11 liblzma/5.2.3 bzip2/1.0.6 liblz4/1.7.5 libzstd/
1.4.5
Found libcurl/7.77.0-DEV Schannel zlib/1.2.11 zstd/1.4.5 libidn2/2.0.4 nghttp/1.31
.0
C:\WINDOWS\system32>
```

หากแสดงเลขเวอร์ชัน แสดงว่าติดตั้งสำเร็จ หากไม่ติดตั้ง Tesseract โปรแกรมยังใช้งานได้ปกติกับ PDF ที่มีข้อความดิจิทัล

8.2 ขั้นตอนที่ 2 เลือก AI ที่จะใช้งาน

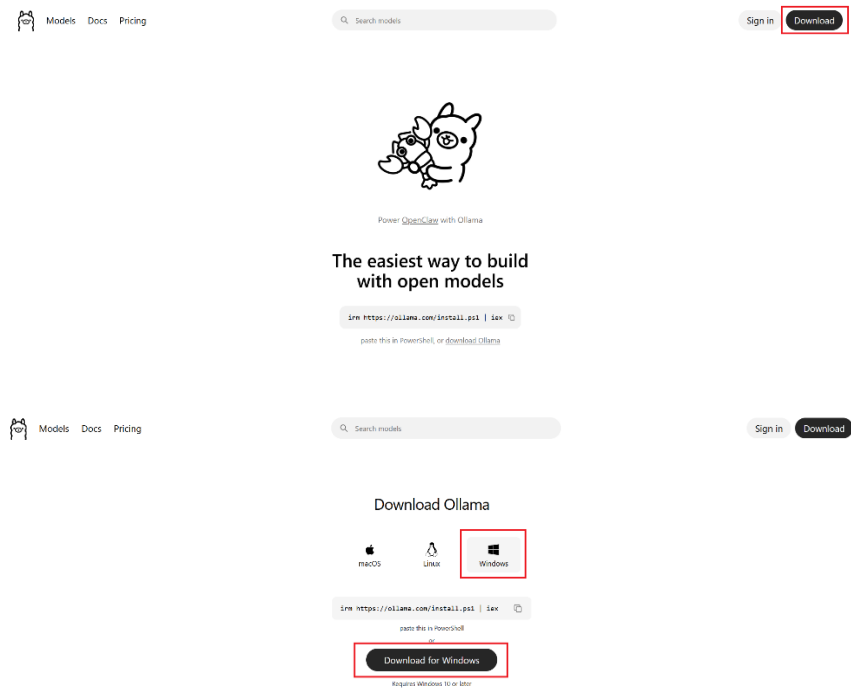
ประเภท	ข้อดี	ข้อเสีย / ข้อจำกัด
Local AI (Gemma 4 ผ่าน Ollama)	ไม่มีค่าใช้จ่ายต่อครั้ง ใช้งานออฟไลน์ได้	ต้องการ RAM ≥ 16 GB และพื้นที่ ~20 GB ทำงานช้า โดยเฉพาะถ้าไม่มี GPU ความแม่นยำต่ำกว่าโมเดล Cloud
EGAT AI Gateway	ตอบเร็ว ใช้งานได้กับคอมสเปคต่ำ	ต้องต่ออินเทอร์เน็ต และต้องอยู่ภายในเครือข่ายของ กฟผ. ต้องใช้ API Key
Cloud AI	ตอบเร็ว ใช้งานได้กับคอมสเปคต่ำ มีโมเดลให้เลือกหลายระดับราคา	ต้องต่ออินเทอร์เน็ต ต้องสมัครและเติมเครดิตล่วงหน้า มีค่าใช้จ่ายตามจำนวน Token

คำแนะนำในการเลือก

- เอกสารลับภายในองค์กรใช้ Local AI
- เอกสารทั่วไป ต้องการความเร็วและความแม่นยำให้ใช้ EGAT AI Gateway หรือ Cloud AI
- คอมพิวเตอร์สเปคต่ำให้ใช้ EGAT AI Gateway หรือ Cloud AI

ติดตั้ง Ollama (เฉพาะกรณีต้องการใช้ Local AI)

- เปิดเบราว์เซอร์ไปที่ <https://ollama.com>
- คลิก "Download" และเลือก Windows



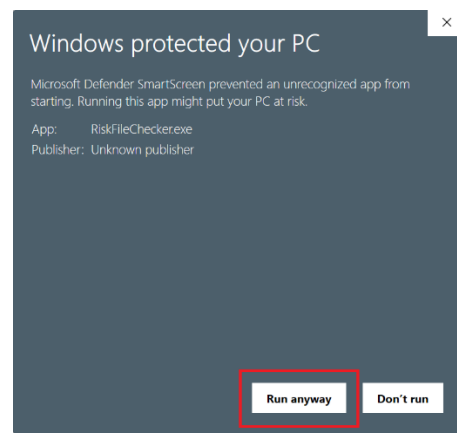
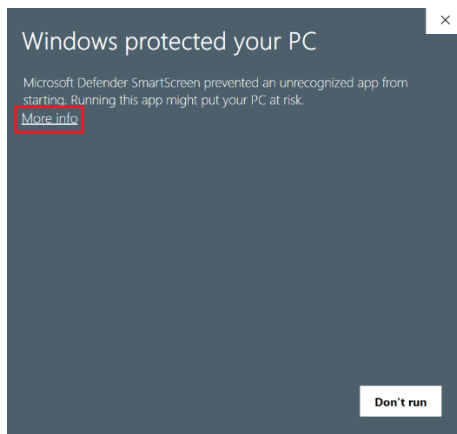
- เปิดไฟล์ OllamaSetup.exe แล้วกด Install



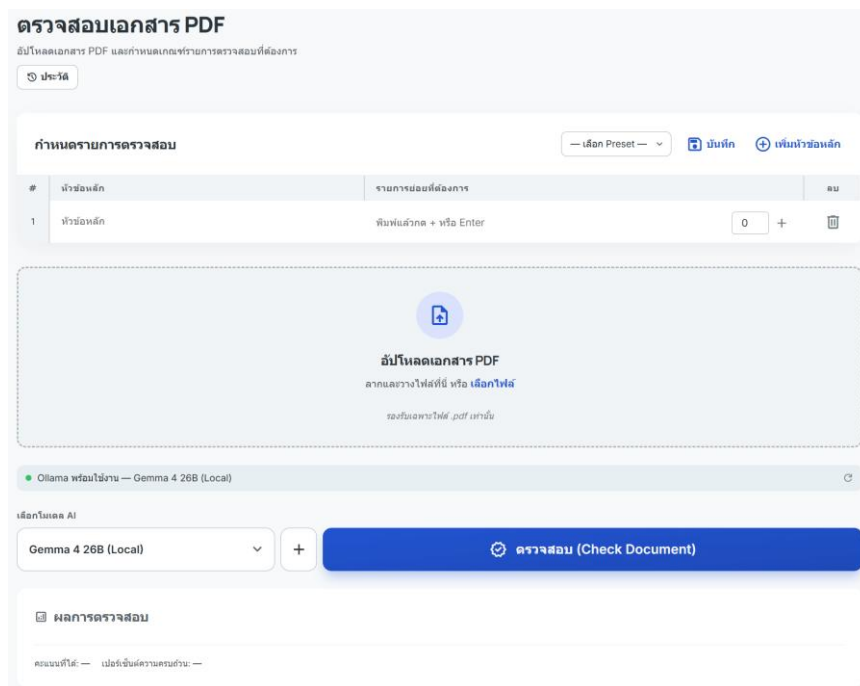
8.3 ขั้นตอนที่ 3 เลือกวิธีติดตั้งโปรแกรม

1. ไปที่หน้า Releases ของ GitHub (<https://github.com/Portrate/EGAT-File-Checker/releases/>)
2. ดาวน์โหลดไฟล์ FileChecker.exe จาก Assets





5. โปรแกรมจะตรวจสอบ Ollama และเปิดเบราว์เซอร์ให้อัตโนมัติ



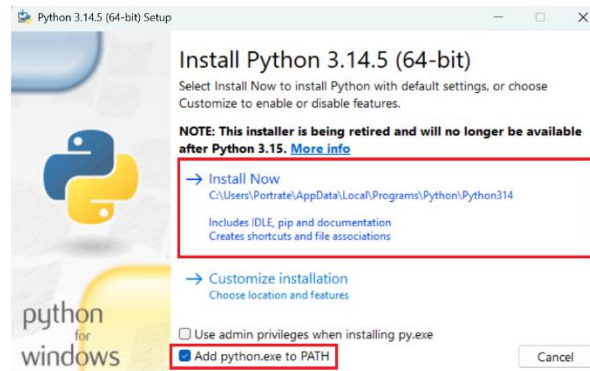
วิธีที่ 2 ใช้ start.bat (ต้องติดตั้ง Python)

ติดตั้ง Python

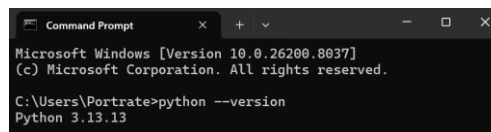
1. เปิดเบราว์เซอร์ไปที่ <https://www.python.org/downloads/>
2. คลิก "Download Python 3.x.x" (เวอร์ชันล่าสุด)



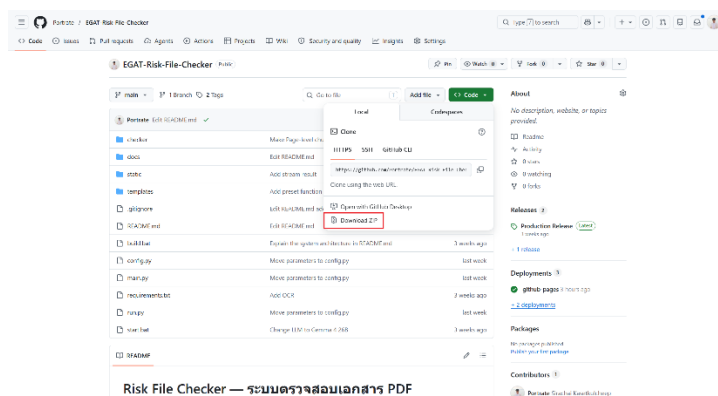
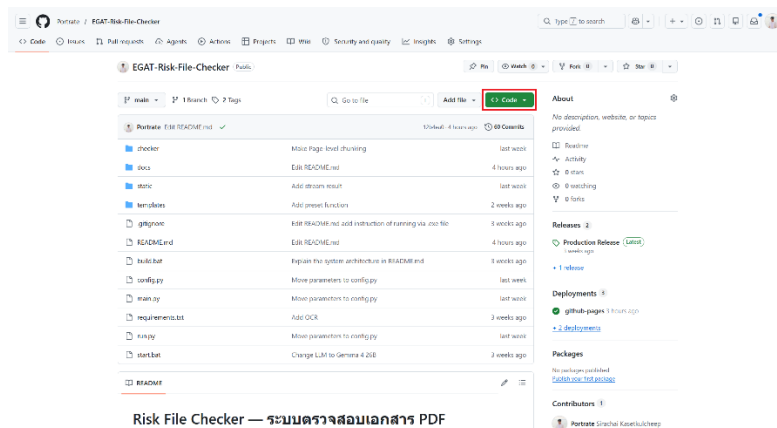
3. เปิดไฟล์ติดตั้ง ดึงถูกที่ "Add Python to PATH" ก่อนกด Install



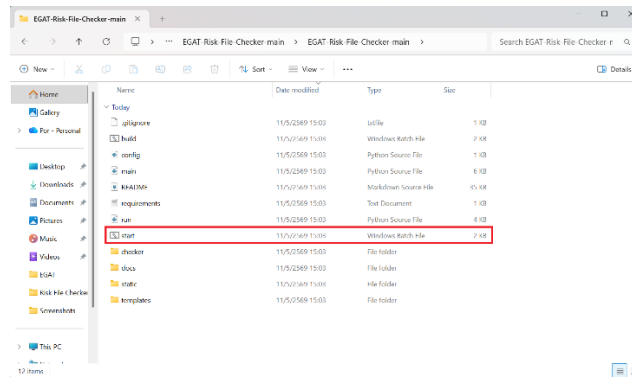
4. ตรวจสอบโดยเปิด Command Prompt แล้วพิมพ์ python --version



5. เปิดหน้า Github ของโปรเจกต์ <https://github.com/Portrate/EGAT-File-Checker>
6. คลิกปุ่มสีเขียว "Code" และ "Download ZIP" แล้วแตกไฟล์



7. ดับเบิลคลิกที่ "start.bat"



- โปรแกรมจะสร้าง environment, ติดตั้ง library, ตรวจสอบ Ollama และเปิดเบราว์เซอร์อัตโนมัติ

9. วิธีใช้งานระบบ

- กรอก Checklist คลิก "เพิ่มหัวข้อ" เพื่อเพิ่มหมวดหมู่ จากนั้นเพิ่มข้อกำหนดย่อยและกำหนดคะแนนแต่ละข้อ สามารถบันทึกหัวข้อเพื่อใช้ในครั้งต่อไปได้

ตรวจสอบเอกสาร PDF
อัปโหลดเอกสาร PDF และกำหนดเกณฑ์การตรวจสอบที่ต้องการ

เลือก Preset: [ปุ่มเพิ่ม] [ปุ่มลบ]

#	หัวข้อหลัก	รายการย่อยที่ตรวจสอบ	คะแนน
1	หัวข้อหลัก	มีหน้าปก หรือ Enter	0

อัปโหลดเอกสาร PDF
สามารถวางไฟล์ได้ หรือ เลือกไฟล์

เลือก AI: Ollama พร้อมใช้งาน — Gemma 4 26B (Local)

เลือกโมเดล AI: Gemma 4 26B (Local) + [ตรวจสอบ (Check Document)]

ผลการตรวจสอบ

สถานะไฟล์: [ปุ่มลบ]

- อัปโหลด PDF ลากไฟล์มาวางในพื้นที่ที่กำหนด หรือคลิกเพื่อเลือกไฟล์

ตรวจสอบเอกสาร PDF
อัปโหลดเอกสาร PDF และกำหนดเกณฑ์การตรวจสอบที่ต้องการ

เลือก Preset: [ปุ่มเพิ่ม] [ปุ่มลบ]

#	หัวข้อหลัก	รายการย่อยที่ตรวจสอบ	คะแนน
1	ตรวจสอบเอกสาร	ตรวจสอบเอกสารว่ามีหน้าปก (2568) หรือไม่ (2568) / ตรวจสอบเอกสารว่ามีหน้าปก 4-2568 หรือไม่ (2568) / ตรวจสอบเอกสารว่ามีหน้าปก 4-2568 หรือไม่ (2568) / ตรวจสอบเอกสารว่ามีหน้าปก 4-2568 หรือไม่ (2568)	0
2	การตรวจสอบเอกสาร	ตรวจสอบเอกสารว่ามีหน้าปก (2568) หรือไม่ (2568) / ตรวจสอบเอกสารว่ามีหน้าปก 4-2568 หรือไม่ (2568) / ตรวจสอบเอกสารว่ามีหน้าปก 4-2568 หรือไม่ (2568)	0

อัปโหลดเอกสาร PDF
สามารถวางไฟล์ได้ หรือ เลือกไฟล์

เลือก AI: Ollama พร้อมใช้งาน — Gemma 4 26B (Local)

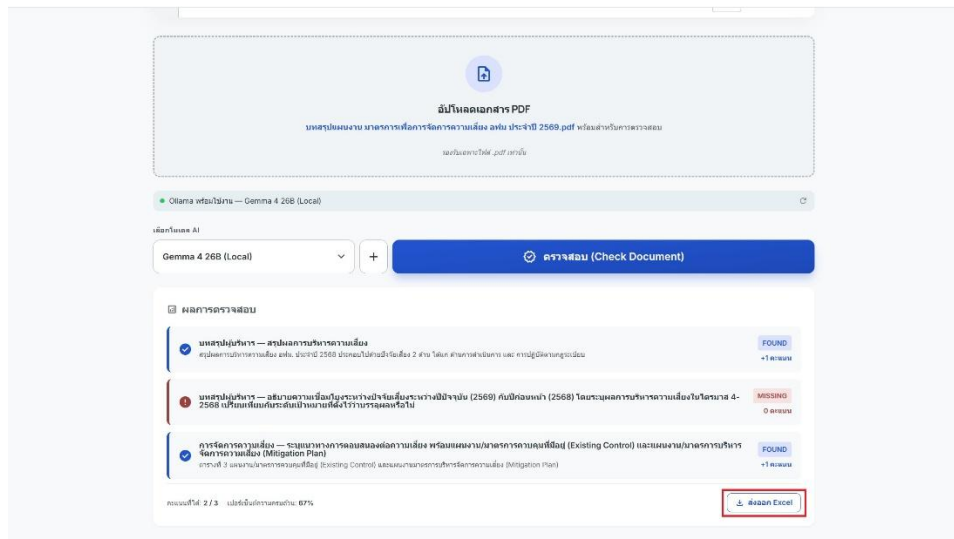
เลือกโมเดล AI: Gemma 4 26B (Local) + [ตรวจสอบ (Check Document)]

- เลือก AI Model เลือกลงจาก Dropdown ถ้าใช้ EGAT AI Gateway หรือ Cloud AI ให้กรอก API Key ด้วย และสามารถเพิ่มโมเดลอื่นๆ เพิ่มเติมได้

- กดปุ่ม "ตรวจสอบ" ระบบจะเริ่มวิเคราะห์และแสดงเวลาที่ผ่านไป

- ดูผลลัพธ์แต่ละข้อแสดงสถานะ ผ่าน/ไม่ผ่าน พร้อมหลักฐานและเหตุผล

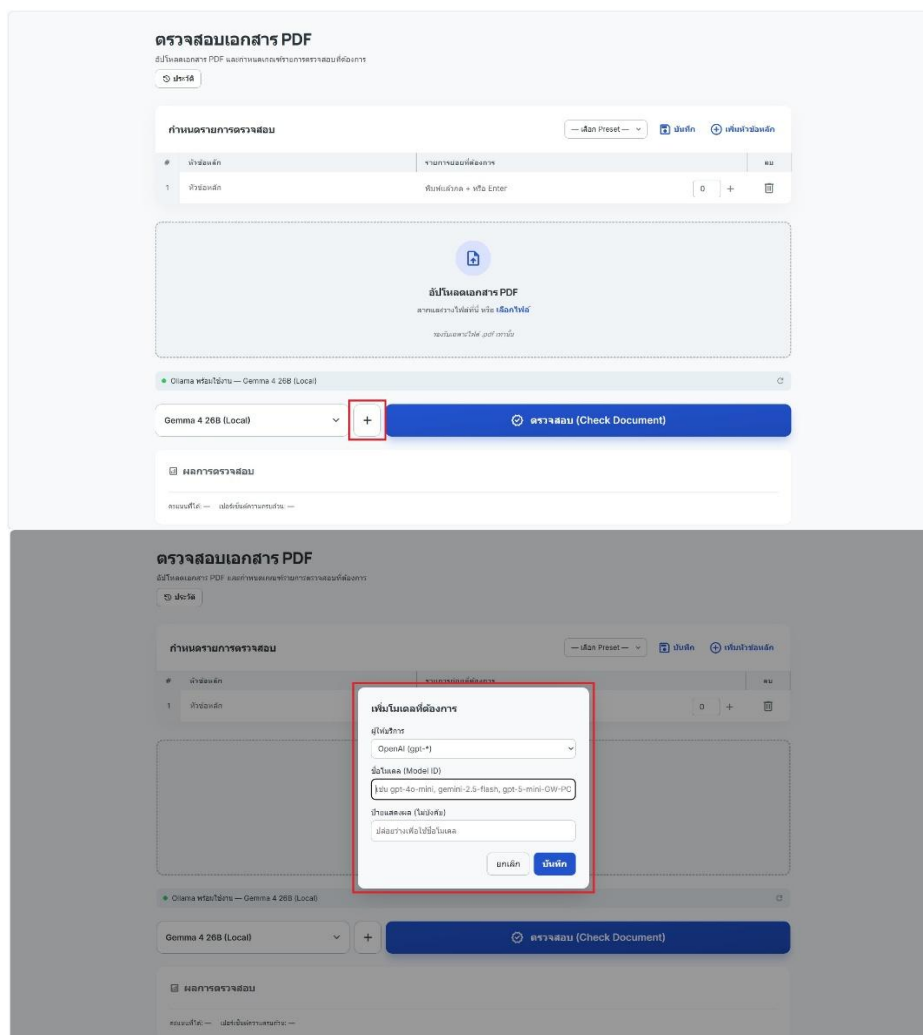
6. ส่งออก Excel คลิปปุ่ม "ส่งออก Excel" เพื่อดาวโหลดรายงาน



การปิดโปรแกรม: กด Ctrl + C ในหน้าต่าง Command Prompt

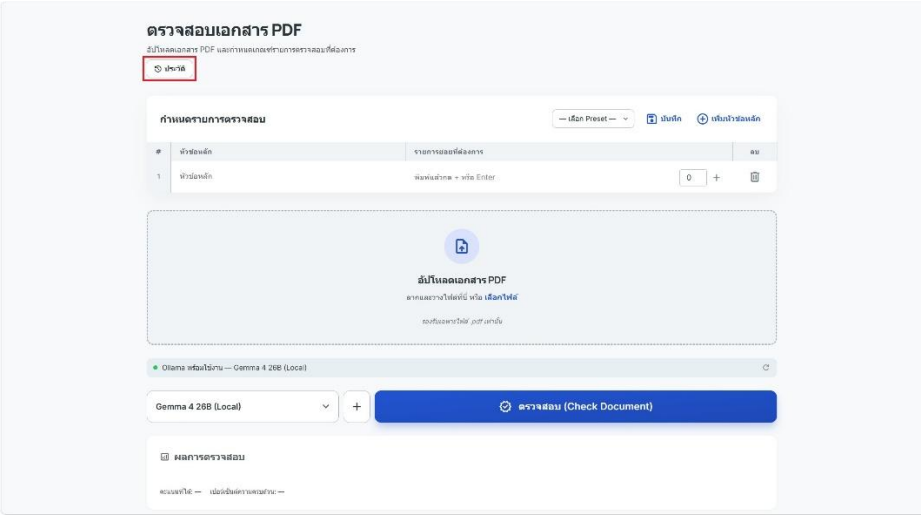
เพิ่มโมเดล AI เองผ่าน UI

ผู้ใช้สามารถเพิ่มโมเดล OpenAI หรือ Gemini รุ่นใหม่ๆ ได้เองโดยไม่ต้องแก้ไขโค้ด ข้อมูลจัดเก็บใน Local Storage เช่นเดียวกับ Preset



ประวัติการตรวจสอบ

ระบบสามารถดูประวัติการตรวจสอบที่ผ่านมาได้ โดยคลิก "ประวัติ"



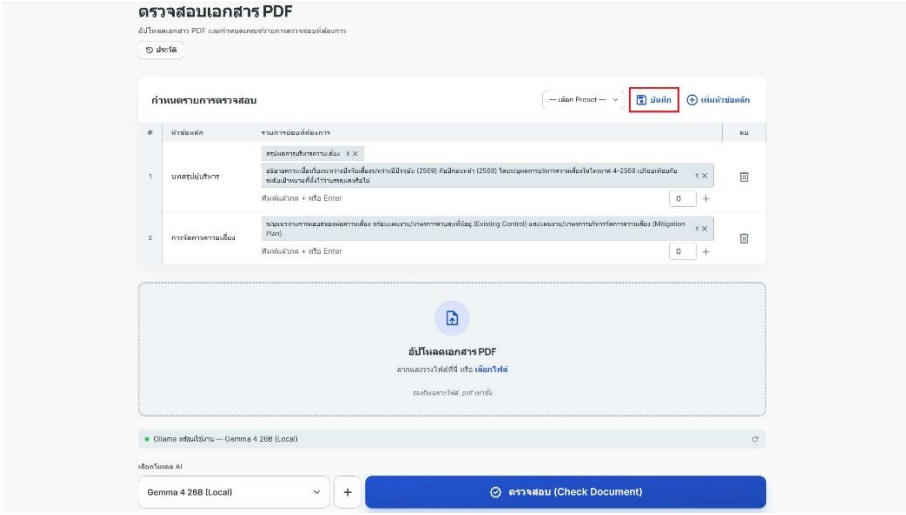
ระบบจะแสดงประวัติการตรวจสอบที่เคยดำเนินการ และสามารถกดเพื่อดูรายละเอียด หรือดาวน์โหลดรายงานได้



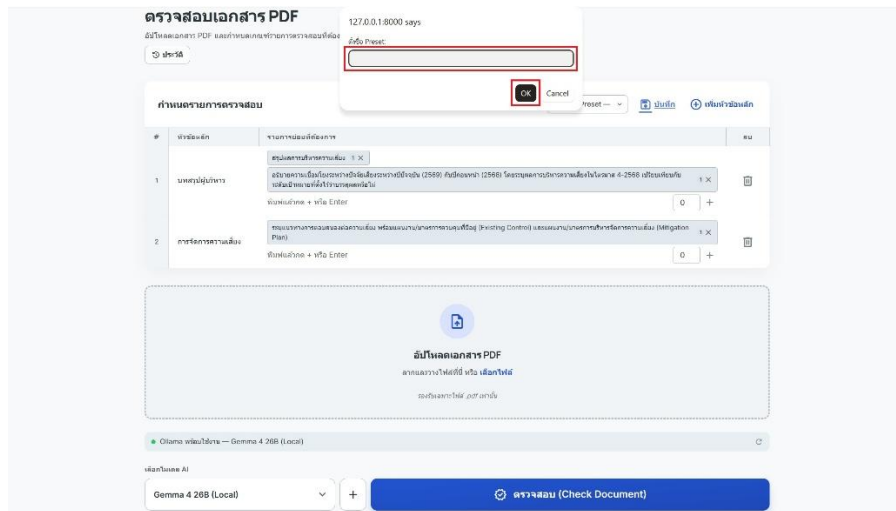
บันทึก Preset ของรายการตรวจสอบ

ระบบสามารถบันทึกรายการตรวจสอบไว้ สำหรับใช้ในการตรวจครั้งถัดไปได้

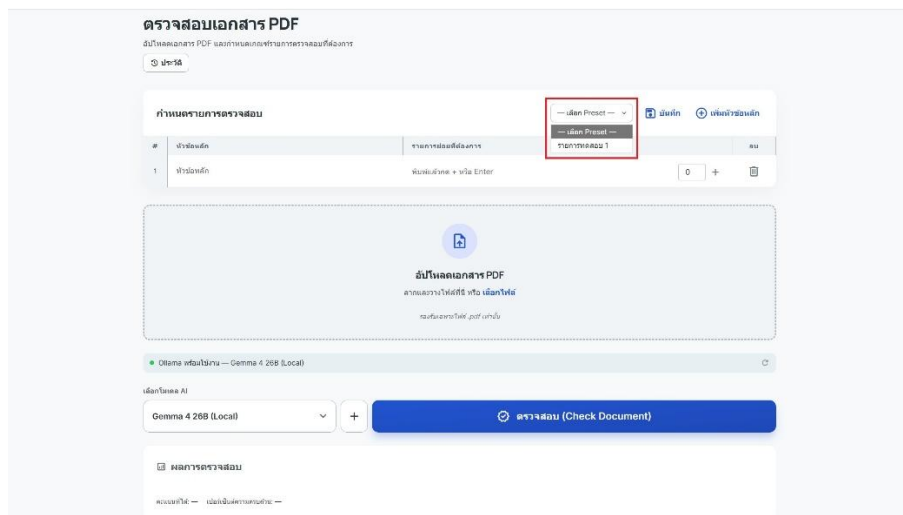
หลังจากกรอกรายการตรวจสอบครบถ้วนแล้วคลิกปุ่ม "บันทึก"



ตั้งชื่อ Preset แล้วคลิกปุ่ม "OK"



เมื่อเปิดระบบขึ้นมาใหม่สามารถนำ Preset รายการตรวจสอบที่ได้บันทึกไว้มาใช้ โดยคลิก Dropdown ที่ "เลือก Preset" และเลือก Preset ที่บันทึกไว้



10. ประสิทธิภาพและข้อจำกัด

10.1 ประสิทธิภาพโดยประมาณ

การดำเนินการ	เวลาโดยประมาณ
อ่าน PDF (ข้อความดิจิทัล)	0.1 – 1 วินาที
อ่าน PDF (OCR ภาพสแกน)	5 – 30 วินาที (ขึ้นกับจำนวนหน้า)
ระยะเวลาวิเคราะห์ต่อข้อของ Local AI (CPU)	มากกว่า 120 วินาที
ระยะเวลาวิเคราะห์ต่อข้อของ Local AI (GPU)	30 – 120 วินาที
ระยะเวลาวิเคราะห์ต่อข้อของ Cloud AI	5 – 30 วินาที
ส่งออก Excel	น้อยกว่า 1 วินาที

10.2 การเปรียบเทียบความเร็วโมเดล (ทดสอบกับเอกสาร GRC กฟผ.)

โมเดล	เวลาที่ใช้ (นาท)
Gemma 4 26B (ทดสอบกับโน้ตบุ๊ก Dell G5 15)	20:26
GPT-5	14:41
GPT-4o	6:32

10.3 ข้อจำกัดที่ควรทราบ

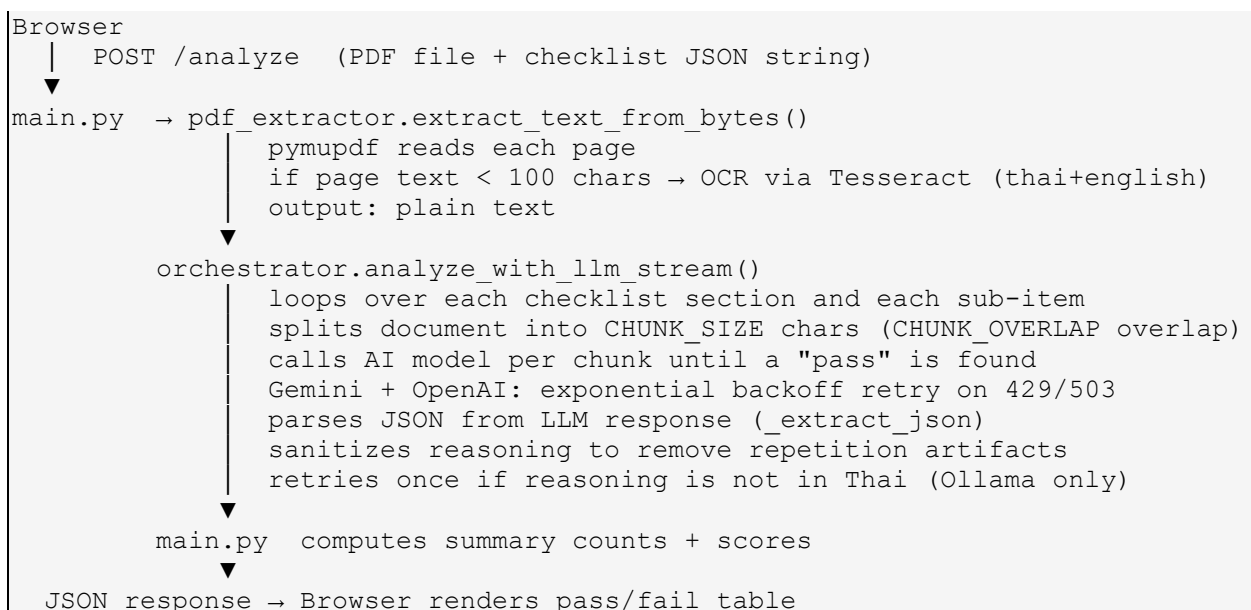
ข้อจำกัด	รายละเอียด
เอกสารขนาดใหญ่	PDF ที่มีขนาดใหญ่ อาจใช้เวลานานมากกับ Local AI
OCR คุณภาพต่ำ	PDF ที่สแกนมาแบบมีความละเอียดต่ำหรือบิดเบี้ยว OCR อาจอ่านผิดพลาด
Context ต่อ Chunk	เอกสารถูกตัดเป็น Chunk ทำให้ AI อาจไม่เห็นภาพรวมทั้งเอกสารในครั้งเดียว

11. การแก้ไขปัญหาเบื้องต้น

ปัญหา	วิธีแก้
Windows เตือน "Protected your PC"	คลิก "More info" และ "Run anyway"
Local AI ไม่พร้อมใช้งาน	ติดตั้ง Ollama และดาวน์โหลดโมเดล AI หรือใช้ Cloud AI แทน
เชื่อมต่อ Ollama ไม่ได้	ตรวจสอบว่ามีไอคอน Ollama ใน System Tray ถ้าไม่มีให้เปิด Ollama ก่อน
หน้าเว็บไม่เปิด	เปิดเบราว์เซอร์แล้วพิมพ์ <code>http://localhost:8000</code> ด้วยตนเอง
ผลลัพธ์แสดงช้ามาก (Local AI)	ควรมี RAM ≥ 16 GB ถ้ามี GPU จะทำงานได้เร็วขึ้น
Port 8000 already in use	รีสตาร์ทเครื่องหรือปิดโปรแกรมอื่นที่ใช้ Port 8000
Python not found (วิธีที่ 2)	ติดตั้ง Python ใหม่โดยติ๊ก "Add Python to PATH"
PDF ที่เป็นภาพสแกนอ่านไม่ออก	ติดตั้ง Tesseract ตามขั้นตอนที่ 6.1 และอย่าลืมติ๊ก "Thai" ระหว่างติดตั้ง
OCR อ่านภาษาไทยไม่ได้	ถอนการติดตั้ง Tesseract แล้วติดตั้งใหม่ โดยติ๊ก "Thai" ใน Additional language data

12. สถาปัตยกรรมโค้ด (Technical Architecture)

12.1 Request Flow



12.2 รายละเอียดไฟล์หลัก

ไฟล์	หน้าที่
config.py	รวบรวมค่าคงที่ทั้งหมด: HOST, PORT, OLLAMA_BASE_URL, DEFAULT_MODEL, CHUNK_SIZE, CHUNK_OVERLAP, MAX_RETRIES ฯลฯ
run.py	Entry point: ตรวจสอบ Ollama, ตรวจสอบโมเดล, เปิดเบราว์เซอร์, ส่งต่อให้ uvicorn
main.py	GET / แสดง index.html, POST /analyze ตรวจสอบ PDF, POST /export/excel ส่งออก xlsx, GET /health ตรวจสอบสถานะ Ollama
checker/pdf_extractor.py	ใช้ PyMuPDF อ่านแต่ละหน้า หน้าที่มีข้อความน้อยกว่า 100 ตัวอักษรจะส่งผ่าน Tesseract OCR
checker/orchestrator.py	Core LLM logic: chunking, การเรียก Ollama/Gemini/OpenAI, retry, JSON parsing, sanitize reasoning
checker/models.py	Pydantic schemas: ChecklistSection, ChecklistItem, ItemResult, SectionResult, AnalysisResponse
checker/excel_exporter.py	สร้างไฟล์ .xlsx ด้วย openpyxl 2 ซีต (สรุปผล และรายละเอียด)
checker/history.py	บันทึกประวัติการตรวจสอบ

12.3 LLM Configuration (config.py)

Parameter	Value	หมายเหตุ
Default model	gemma4:26b	เปลี่ยนได้ใน config.py หรือเลือกผ่าน UI
Context window	32,768 tokens	num_ctx ใน OLLAMA_OPTIONS (Ollama only)
Max output tokens	256	num_predict — enough for one JSON object
Temperature	0	deterministic output
Chunk size	24,000 chars	CHUNK_SIZE ใน config.py
Chunk overlap	2,000 chars	CHUNK_OVERLAP ใน config.py
Read timeout (Ollama)	1,200 s (20 min)	OLLAMA_TIMEOUT ใน config.py
Read timeout (Cloud)	120 s	CLOUD_TIMEOUT ใน config.py
Max retries (Cloud)	5	MAX_RETRIES ใน config.py

13. รายการ Dependencies

13.1 External Software (ติดตั้งแยก)

- Ollama Local AI runtime (ถ้าต้องการใช้ Local AI)
- Tesseract OCR OCR engine พร้อม Language Pack ภาษาไทย (ถ้าต้องการอ่าน PDF ภาพสแกน)

13.2 Python Libraries (requirements.txt)

Library	หน้าที่
FastAPI + Uvicorn	Backend web framework
PyMuPDF (fitz)	อ่านข้อความจาก PDF
pytesseract + Pillow	OCR สำหรับ PDF ที่เป็นภาพสแกน
httpx	Async HTTP client สำหรับเรียก AI API
Pydantic v2	Data validation
openpyxl	สร้างไฟล์ Excel
Jinja2	Template engine สำหรับ HTML
PyInstaller	แพ็คเกจโปรแกรมเป็นไฟล์ .exe

14. สรุปและแนวทางการพัฒนาต่อ ยอด

14.1 สิ่งที่ระบบทำได้

- ตรวจสอบเอกสาร PDF อัตโนมัติด้วย AI
- รองรับทั้งเอกสารดิจิทัลและภาพสแกน (OCR)
- เลือก AI ได้ตามความเหมาะสมของเอกสาร (Local / EGAT AI Gateway / Cloud AI)
- แสดงผลทันทีแบบ Real-time พร้อมคะแนนและหลักฐานอ้างอิง
- ส่งออกรายงาน Excel ได้ทันที
- ติดตั้งง่าย ใช้ไฟล์ EXE ไฟล์เดียว ไม่ต้องติดตั้ง Python

14.2 แนวทางการพัฒนาต่อ ยอด

- รองรับการตรวจสอบหลายไฟล์พร้อมกัน
- เพิ่มระบบจัดเก็บประวัติการตรวจสอบ
- พัฒนาเป็นระบบบนเว็บกลางขององค์กร
- เพิ่มการเปรียบเทียบผลระหว่างโมเดล AI
- รองรับไฟล์ประเภทอื่น เช่น Word, PowerPoint